

P1494R5: Partial program correctness

Audience: CWG; LWG

S. Davis Herring <herring@lanl.gov>

Los Alamos National Laboratory

February 14, 2025

History

Since [r4](#):

- Rebased onto N5001
- Moved to <utility>
- Extended wording to cover more I/O cases

Since [r3](#):

- Rebased onto N4988
- Made I/O automatically a checkpoint
- Added missing erroneous behavior context
- Added C cross reference for host environment
- Removed outdated note
- Clarified, simplified, and corrected wording

Since [r2](#):

- Rebased onto N4981
- Required freestanding support, per LEWG review
- Clarified implementability
- Mentioned implicit checkpoint alternative

Since [r1](#):

- Discussed the preexisting problem of input dependence
- Discussed interactions with other kinds of observable behavior
- Discussed lack of debugger support
- Fixed (preexisting) “first” in wording based on SG1 review

Since [r0](#):

- Made introduction explicitly independent of contracts
- Fixed conditionals in example

Problem

Undefined behavior enables and extends many important optimizations (*e.g.*, simplifying signed integer arithmetic and dead-code elimination). The “time travel” aspect of such optimizations (explicitly authorized by [intro.abstract]/5) is surprising to many programmers in that it can sometimes eliminate tests meant to detect the invalid operation in question. In particular, consider

```
#include<cstdio>
#include<cstdlib>

static void bad(const char *msg) {
    std::fputs(msg, stderr);
#ifdef DIE
    std::abort();
#endif
}

void inc(int *p) {
    if(!p) bad("Null!\n");
    ++*p;
}
```

Without `-DDIE`, a conforming implementation can elide the test in `inc` entirely: `std::fputs` always returns, so any call `inc(nullptr)` is guaranteed to have undefined behavior and need not call `bad`. (Note that current implementations do not do so in this case.)

This issue came up again near the beginning of 2019 in the discussion of contracts:

```
void f(int *p) [[expects: p]] [[expects: *p<5]];
```

Discomfort with the idea that (with continuation mode “on”) the first *contract-attribute-specifier* might be elided because of the second was one of the motivations for the many late proposals to change (and eventually remove) contracts. [Many wondered](#) about the possibility of making a contract violation handler “opaque to optimization”, so that the first precondition must be checked on the supposition that the handler might not return (but rather throw or terminate).

The capability of establishing such a “checkpoint”, where subsequent program behavior, *even if undefined*, does not affect the preceding behavior, would be useful in general for purposes of stability and debugging.

There is already an analogous issue concerning program input: clearly a program can have undefined behavior for some inputs and not others. [intro.abstract]/3 and /5 acknowledge this by referring to “a given input” and “that input”, but such a monolithic approach neglects the paradoxical possibility of input correlated with, say, unspecified quantities:

```
int x[1];

int main() {
    std::uintptr_t a=reinterpret_cast<std::uintptr_t>(x)%3+1,b;
```

```

std::cerr << "The car is behind door number " << a
          << ". Door to open: ";
std::cin >> b;
return x[a-b];
}

```

This program might suggest 2 (if the cast yields, say, 0x1000), but one could absurdly argue that responding with 2 leads to undefined behavior because almost all of the unspecified possibilities for the “address” of `x` (e.g., 0x2000) lead to undefined behavior for that input. An OOTA interpretation is also available: “`a` is really 3, so the input 2 leads to undefined behavior, whose effect is to print 2 instead of `a`”. We should reject this on the grounds of causality, since we require that undefined behavior respect input which can quite reasonably depend on *prior* output.

The standard does not have the necessary notion of *prior*, and this paper does not address this situation, but its checkpoints may be used to strengthen the guarantee of /7.3, that prompts are delivered before waiting for input, to include all observable behavior. This usage does exclude the OOTA interpretation: the output delivered to the host environment must accurately represent a if (as is automatically true) it is separated by a checkpoint from any potential undefined behavior.

Previous work

I [suggested](#) a trick involving a `volatile` variable, based on the idea that a volatile read is *observable behavior* ([intro.abstract]/7) that must be preserved by optimization.

```

inline void log(const char *msg)
{std::fputs(msg, stderr);}    // always returns

bool on_fire() {
    static volatile bool fire; // always false
    return fire;
}

void f(int *p) {
    if (p == nullptr) log("bad thing 1");
    if (on_fire()) std::abort();
    if (*p >= 5) log("bad thing 2");
}

```

The idea is that the compiler cannot assume that `on_fire()` returns `false`, and so the check for `p` being null cannot be eliminated. However, the compiler can observe that, if `p` is null, the behavior will be undefined *unless* `on_fire()` returns `true`, and so it can elide that check (though not the volatile read) and call `abort()`. This therefore seems to convey a certain capability of *observing* the upcoming undefined behavior without actually experiencing it.

Unfortunately, conforming implementations are not constrained to follow this analysis. It is logically necessary that the implementation perform the observable volatile read unless it can somehow obtain its result otherwise. However, after reading the value `false` (as of course it will be in practice) the implementation may take any action whatsoever, even

“undoing” the call to `log`. For example, it would be permissible to perform the implicit flush for `stderr` only just before the call to `std::abort` (which never happens). One might hope for the implementation to allow for the possibility that `log` affects some hardware state that affects the volatile read, but it might not as such a scheme would require support from the operating system.

General solution

We can instead introduce a special library function

```
namespace std {  
    // in <utility>  
    void observable() noexcept;  
}
```

that divides the program’s execution into *epochs*, each of which has its own observable behavior. If any epoch completes without undefined behavior occurring, the implementation is required to exhibit the epoch’s observable behavior. Ending an epoch is nonetheless distinct from ending the program: for example, there is no automatic flushing of `<cstdio>` streams.

Undefined behavior in one epoch may obscure the observable behavior of a previous epoch (for example, by re-opening an output file), but external mechanisms such as pipes to a logging process can be used to guarantee receipt of an epoch’s output. With multiple threads, it is not the epochs themselves that are meaningful but their boundaries (or *checkpoints*); normal thread synchronization is required for the observable behavior of one thread to be included in an checkpoint defined by another.

As a practical matter, a compiler can implement `std::observable` efficiently as an intrinsic that counts as a possible termination, which the optimizer thus cannot remove. After optimization (including any link-time optimization), the code generator can then produce zero machine instructions for it. In Tokyo, some concerns were raised regarding implementability of this proposal, perhaps because of what then appeared to be dependence on the library function; the wording has been adjusted to avoid that implication. Discussions there and since with Michael Spencer suggest that the implementation strategy outlined in this paragraph is practicable.

Note that `std::observable` does not itself constitute observable behavior, and it does not forgive infinite non-trivial empty loops ([intro.progress]/1). There is no explicit connection to volatile access, but the ordinary happens-before rules apply (as much as possible given the vacuous [intro.abstract]/7.1). Finally, there is no guarantee that, for instance, local variables have been spilled to registers at each checkpoint:

`std::observable` prevents certain program reorderings, but it is not a general aid for comprehensibility when using a debugger. (In general, it is difficult if not impossible to specify semantics that allow optimization and yet behave correctly when presented with input from a user equipped with a debugger.)

Limited assumptions

A call to `std::observable` prevents the propagation of assumptions based on the potential for undefined behavior after it into code before it. The following functions (ignoring for illustration the implicit checkpoints associated with the I/O functions) offer the same opportunities for dead-code elimination:

```
void a(int &r, int *p) {  
    if (!p)  
        std::fprintf(stderr,  
            "count: %d\n",  
            ++r);  
    if (!p) std::abort(); //  
        henceforth, p is  
        known to be non-null  
    if (!p)  
        std::fprintf(stderr,  
            "p is null\n");  
}
```

```
void b(int &r, int *p) {  
    if (!p)  
        std::fprintf(stderr,  
            "count: %d\n",  
            ++r);  
    std::observable();  
    if (!p)  
        std::fprintf(stderr,  
            "p is null\n");  
    *p += r; //  
        p may be assumed  
        non-null  
}
```

In both cases, the “p is null” output can be elided: in a, because execution would not continue past the `std::abort`; in b, because of the following dereference of p. In both cases, the count output must appear if p is null: in a, because the program thereafter has the defined behavior of aborting; in b, because the epoch ends before undefined behavior occurs.

The function b, however, offers the additional optimization of not checking for null pointers at run time. It is very useful to support such optimizations without compromising diagnostics.

Usage

The obvious place to use `std::observable` would be after any sort of I/O that always returns, especially in any code run when an error is detected (and so imminent undefined behavior is likely). In a contracts context, the violation handler is one such routine; since `std::observable()` has no side effects, it would also be permissible to include it in specific contract conditions to guarantee that previous contracts are checked (even if the violation handler always returns):

```
void f(int *p) pre(p) pre((std::observable(), *p<5));
```

The [current Contracts paper](#) inserts appropriate observable checkpoints in the sequence of assertion evaluations automatically rather than requiring the above syntax.

Implicit approach

In C23, all I/O (including volatile accesses) is implicitly considered an observable checkpoint (albeit without that phrasing, instead [changing the definition of undefined behavior](#)). (Formally this refers to `fflush` and not `printf`, but the latter calls the former on a schedule that the optimizer cannot in general predict.) The wording below includes small changes to make all library I/O consistent in this regard.

Because even future versions of C are likely not to need manual checkpoints, there is no need to provide `::observable` in a C header. The most appropriate C++ header appears to be `<utility>`, where `std::observable` can be a counterpart to `std::unreachable`.

An objection might be raised to this approach that the compiler doesn't know the semantics of every function that performs I/O (and we can't expect, say, POSIX to annotate its definition of `write` with C++-specific text). However, such a situation is harmless: any function whose definition is truly unknown to the implementation might terminate the program, so no optimization can propagate back past a call to such a function anyway. If the compiler does know about a platform-specific I/O function (and, say, that it always returns), it is no more complicated for it to know that it is an optimization barrier as well.

The other plausible objection is that optimization might be too strongly affected by such ordering, although the I/O itself would tend to drown such savings. In August 2024, LEWG found these concerns not to be problematic and asked for library I/O to be an automatic checkpoint and asked for EWG to similarly consider the case of volatile accesses (where optimization might be more of a concern). If volatile accesses were also made observable, it would make sense to adopt the requisite changes to the definition of undefined behavior (and automatic checkpoints) without the user-accessible `std::observable`. In Wrocław, EWG considered the `volatile` question, but did not reach consensus on any change.

Wording

Relative to N5001.

#[intro.abstract]

Remove the note in paragraph 4:

- Certain other operations are described in this document as undefined behavior (for example, the effect of attempting to modify a `const` object).

[Note: This document imposes no requirements on the behavior of programs that contain undefined behavior. — *end note*]

Insert before paragraph 5:

- Certain events in the execution of a program are termed *observable checkpoints*.

[Note: A call to `std::observable` ([utility.undefined]) is an observable checkpoint. — end note]

Change paragraph 5:

- The *defined prefix* of an execution comprises the operations O for which for every undefined operation U there is an observable checkpoint C such that O happens before C and C happens before U .

[Note: The undefined behavior that arises from a data race ([intro.races]) occurs on all participating threads. — end note]

A conforming implementation executing a well-formed program shall produce the ~~same~~ observable behavior as of the defined prefix of one of the possible executions of the corresponding instance of the abstract machine with the same program and the same input. ~~However, if any such the selected~~ execution contains an undefined operation, ~~this document places no requirement on~~ the implementation executing that program with that input ~~(not even with regard to operations preceding the first undefined operation)~~ may produce arbitrary additional observable behavior afterwards. If the execution contains an operation specified as having erroneous behavior, the implementation is permitted to issue a diagnostic and is permitted to terminate the execution at an unspecified time after that operation.

Change paragraph 7:

- The ~~least requirements on a conforming implementation are~~ following specify the observable behavior of the program:

1. Accesses through volatile glvalues are evaluated strictly according to the rules of the abstract machine.
2. ~~At program termination, all d~~ Data is delivered to the host environment to be written into files shall be identical to one of the possible results that execution of the program according to the abstract semantics would have produced (C17 7.21.3).

[Note: Delivering such data is followed by an observable checkpoint ([cstdio.syn]). Not all host environments provide access to file contents before program termination. — end note]

3. The input and output dynamics of interactive devices shall take place in such a fashion that prompting output is actually delivered before a program waits for input. What constitutes an interactive device is implementation-defined.

~~These collectively are referred to as the observable behavior of the program.~~

[Note: More stringent correspondences between abstract and actual semantics can be defined by each implementation. — end note]

#[utilities]

#[utility.syn]

Change part of the synopsis:

```
- | // [utility.unreachable undefined], unreachable undefined behavior  
| [[noreturn]] void unreachable();  
| void observable() noexcept;
```

#[utility.unreachable undefined] Function unreachable Undefined behavior

Rename subclause and change stable label (directing the old to the new).

Add at the end of the subclause:

```
- | void observable() noexcept;  
- | Effects: Establishes an observable checkpoint ([intro.abstract]).
```

#[input.output]

#[iostream.objects.overview]

Change paragraph 2:

```
- | The header <iostream> declares objects that associate objects with the standard  
| C streams provided for by the functions declared in <cstdio>, and includes all the  
| headers necessary to use these objects. The dynamic types of the stream buffers  
| initially associated with these objects are unspecified, but they have the behavior  
| specified for std::basic_filebuf<char> or std::basic_filebuf<wchar_t>.
```

#[iostream.format]

#[ostream.formatted.print]

Change bullet (4.3):

```
- | If the function is vprint_unicode and os is a stream that refers to a terminal that  
| is capable of displaying Unicode only via a native Unicode API, which is  
| determined in an implementation-defined manner, flushes os and then writes out  
| to the terminal using the native Unicode API; if out contains invalid code units,  
| the behavior is undefined. Then establishes an observable  
| checkpoint ([intro.abstract]).
```

#[print.fun]

Change bullet (10.1):

- If `stream` refers to a terminal that is capable of displaying Unicode only via a native Unicode API, flushes `stream` and then writes out to the terminal using the native Unicode API; if `out` contains invalid code units, the behavior is undefined. Then establishes an observable checkpoint ([intro.abstract]).

#[filebuf.virtuals]

Change paragraph 10:

- 3. [...]
 4. Otherwise output from `xbuf` to `xbuf_end`, and fail if output fails. At this point if `b != p` and `b == end` (`xbuf` isn't large enough) then increase `XSIZE` and repeat from the beginning.Then establishes an observable checkpoint ([intro.abstract]).

#[cstdio.syn]

Insert before paragraph 2:

- The return from each function call that delivers data to the host environment to be written to a file (C17 7.21.3) is an observable checkpoint ([intro.abstract]).